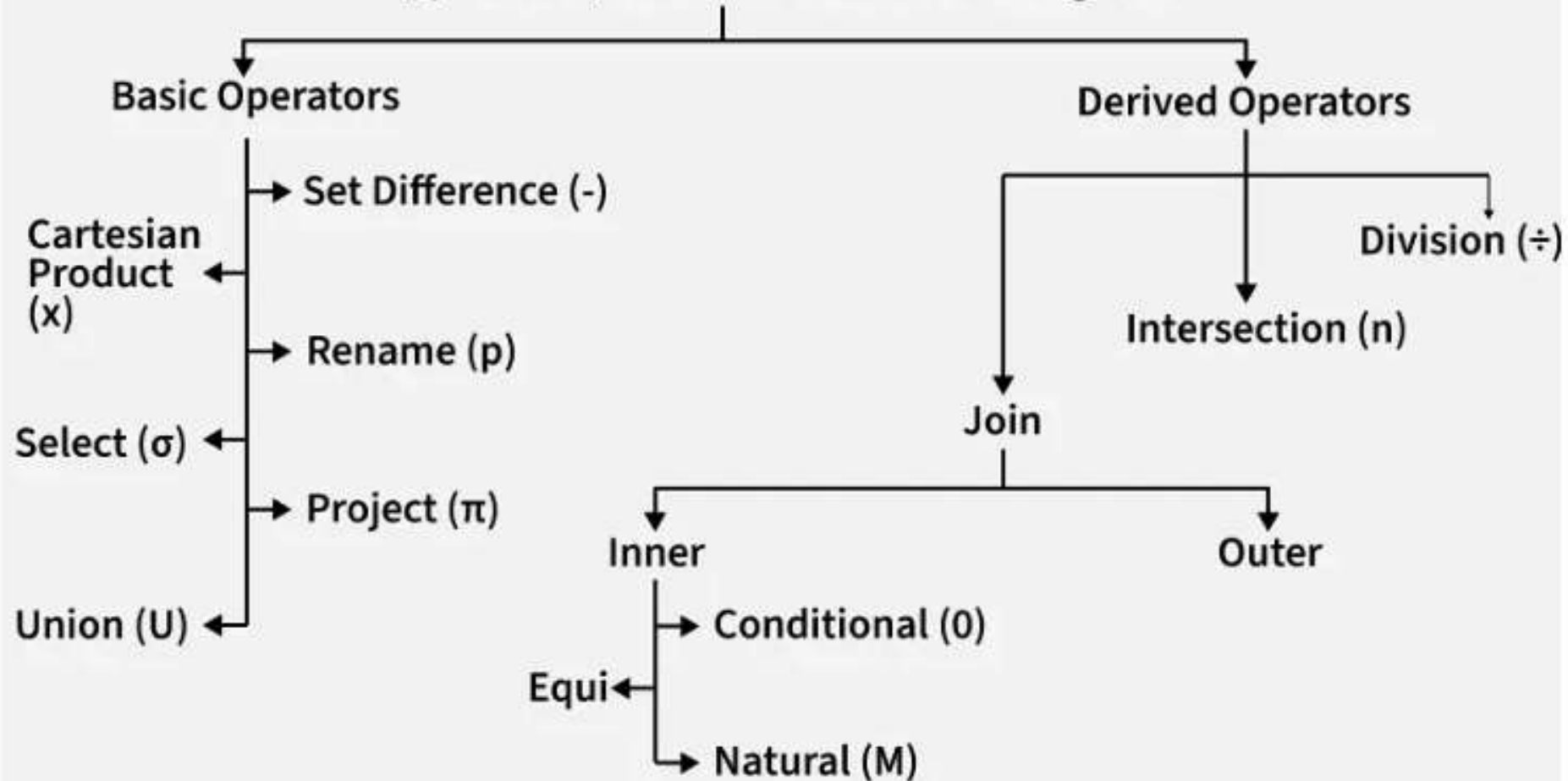


Relational Algebra

RELATIONAL ALGEBRA

- The basic set of operations for the formal relational model is the **relational algebra**.
- These operations enable a user to specify basic retrieval requests as relational algebra expressions.
- The result of the retrieval query will be a new relation.

Types of Operators in Relational Algebra



UNARY RELATIONAL OPERATIONS: SELECT

- The SELECT operation (denoted by σ (sigma)) is used to select a *subset* of the tuples from a relation based on a **selection condition**.
 - The selection condition acts as a **filter**
 - Keeps only those tuples that satisfy the qualifying condition
 - Tuples satisfying the condition are *selected* whereas the other tuples are discarded (*filtered out*)

▪ Examples:

1. Select the EMPLOYEE tuples whose department number is 4:

$$\sigma_{\text{DNO} = 4} (\text{EMPLOYEE})$$

2. Select the employee tuples whose salary is greater than \$30,000:

$$\sigma_{\text{SALARY} > 30,000} (\text{EMPLOYEE})$$

■ **Example:**

1. Select the tuples for all employees who either work in department 4 and make over 25,000 per year, or work in department 5 and make over 50,000 per year.

Solution:

$\sigma_{(Dno=4 \text{ and Salary} > 25,000) \text{ or } (Dno=5 \text{ and Salary} > 50,000)} (EMPLOYEE)$

UNARY RELATIONAL OPERATIONS: PROJECT

- PROJECT Operation is denoted by π (**pi**)
- This operation keeps certain *columns* (attributes) from a relation and discards the other columns.
- PROJECT creates a vertical partitioning
 - The list of specified columns (attributes) is kept in each tuple
 - The other attributes in each tuple are discarded

- **Example:** List each employee's first and last name and salary

$$\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE})$$

- The general form of the *project* operation is:

$$\pi_{\langle \text{attribute list} \rangle}(\mathbf{R})$$

- π (pi) is the symbol used to represent the *project* operation
- $\langle \text{attribute list} \rangle$ is the desired list of attributes from relation R.
- The output of the PROJECT operation has only the attributes specified in $\langle \text{attribute list} \rangle$ in the same order as they appear in the list.

- **Example:** Retrieve the first name, last name, and salary of all employees who work in the department number 5.

Solution:

$$\pi_{\text{FNAME, LNAME, SALARY}}(\sigma_{\text{Dno=5}}(\text{EMPLOYEE}))$$

OR

- We can explicitly show the *sequence of operations*, giving a name to each intermediate relation:

$$\begin{aligned}\text{DEP5_EMPS} &\leftarrow \sigma_{\text{DNO=5}}(\text{EMPLOYEE}) \\ \text{RESULT} &\leftarrow \pi_{\text{FNAME, LNAME, SALARY}}(\text{DEP5_EMPS})\end{aligned}$$

UNARY RELATIONAL OPERATIONS: RENAME

- The RENAME operator is denoted by ρ (**rho**)
- In some cases, we may want to *rename* the attributes of a relation or the relation name or both
- Useful when a query requires multiple operations.

- The general RENAME operation ρ can be expressed by any of the following forms:
 - $\rho_S(B_1, B_2, \dots, B_n)(R)$ changes both:
 - the relation name to S, *and*
 - the column (attribute) names to $B_1, B_1, \dots, B_n$
 - $\rho_S(R)$ changes:
 - the *relation name* only to S
 - $\rho_{(B_1, B_2, \dots, B_n)}(R)$ changes:
 - the *column (attribute) names* only to $B_1, B_1, \dots, B_n$

RELATIONAL ALGEBRA OPERATIONS FROM SET THEORY: INTERSECTION

- INTERSECTION is denoted by $\cap$
- The result of the operation $R \cap S$, is a relation that includes all tuples that are in both R and S
 - The attribute names in the result will be the same as the attribute names in R
- The two operand relations R and S must be “type compatible”

- **Example:** Retrieve the social security numbers of all employees who work in department 5 and directly supervise an employee who works in department 5.

Solution:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5} (\text{EMPLOYEE})$$
$$\text{RESULT1} \leftarrow \pi_{\text{SSN}}(\text{DEP5_EMPS})$$
$$\text{RESULT2}(\text{SSN}) \leftarrow \pi_{\text{SUPERSSN}}(\text{DEP5_EMPS})$$
$$\text{RESULT} \leftarrow \text{RESULT1} \cap \text{RESULT2}$$

- The intersection operation produces the tuples that are in both RESULT1 and RESULT2.

EXAMPLE:

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

UNION: STUDENT $\cup$ INSTRUCTOR

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

INTERSECTION: STUDENT $\cap$ INSTRUCTOR

Fn	Ln
Susan	Yao
Ramesh	Shah

SET DIFFERENCE: STUDENT – INSTRUCTOR

F _n	L _n
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

SOME PROPERTIES OF UNION, INTERSECT, AND DIFFERENCE

- Both Union and Intersection are *commutative* operations; that is
 - $R \cup S = S \cup R$, and $R \cap S = S \cap R$
- Both Union and Intersection can be treated as n-ary operations applicable to any number of relations as both are *associative* operations; that is
 - $R \cup (S \cup T) = (R \cup S) \cup T$
 - $(R \cap S) \cap T = R \cap (S \cap T)$
- The Minus operation is not commutative; that is, in general
 - $R - S \neq S - R$

- In SQL there are three operations UNION, INTERSECT and EXCEPT that correspond to the set operation UNION, INTERSECT and MINUS.
- In addition to these, there are multi-set operations UNION ALL, INTERSECT ALL and EXCEPT ALL that do not eliminate duplicates.
- INTERSECTION can be expressed in terms of union and set difference as follows:

$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$

Example

A

10

10

20

30

B

10

20

Using Except we get answer=30

Using Except all we get answer= 10 30

Cartesian Product (X)

This is used to combine data from two different relations (tables) into one and fetch data from the combined relation.

Syntax: $A \times B$

For example, if we want to find the information for Regular Class and Extra Class which are conducted during morning, then, we can use the following operation:

```
 $\sigma_{\text{time} = \text{'morning'}} (\text{RegularClass} \times \text{ExtraClass})$ 
```

For the above query to work, both **RegularClass** and **ExtraClass** should have the attribute **time**.

STUDENT

SID	Name
-----	------

1	Anu
---	-----

2	Ravi
---	------

COURSE

CID	Course
-----	--------

C1	DBMS
----	------

C2	OS
----	----



Result

SID	Name	CID	Course
-----	------	-----	--------

1	Anu	C1	DBMS
---	-----	----	------

1	Anu	C2	OS
---	-----	----	----

2	Ravi	C1	DBMS
---	------	----	------

2	Ravi	C2	OS
---	------	----	----

- **Example :** Retrieve the of names of each female employee's dependents.

$$\text{FEMALE_EMPS} \leftarrow \sigma_{\text{SEX}='F'}(\text{EMPLOYEE})$$

$$\text{EMP_NAMES} \leftarrow \pi_{\text{FNAME, LNAME, SSN}}(\text{FEMALE_EMPS})$$

$$\text{EMP_DEPENDENTS} \leftarrow \text{EMP_NAMES} \times \text{DEPENDENT}$$

- EMP_DEPENDENTS will contain every combination of EMP_NAMES and DEPENDENT
 - whether or not they are actually related

Figure 5.6

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNAMES

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

EMP_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNAMES

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

EMP_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...

ACTUAL_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...

RESULT

Fname	Lname	Dependent_name
Jennifer	Wallace	Abner

BINARY RELATIONAL OPERATIONS: JOIN

▪ JOIN Operation

- The sequence of CARTESIAN PRODECT followed by SELECT is used quite commonly to identify and select related tuples from two relations
- A special operation, called JOIN combines this sequence into a single operation
- This operation is very important for any relational database with more than a single relation, because it allows us *combine related tuples* from various relations
- The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is:

$$R \bowtie_{\langle \text{join condition} \rangle} S$$

- where R and S can be any relations that result from general *relational algebra expressions*.



- **Example:** Suppose that we want to retrieve the name of the manager of each department.

- To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple.
- We do this by using the join $\bowtie$ operation.

- $\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie \text{EMPLOYEE}$ MGRSSN=SSN

- MGRSSN=SSN is the join condition
- Combines each department record with the employee who manages the department
- The join condition can also be specified as DEPARTMENT.MGRSSN= EMPLOYEE.SSN

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Figure 6.6

Result of the JOIN operation

SOME PROPERTIES OF JOIN

- Consider the following JOIN operation:
 - $R(A_1, A_2, \dots, A_n) \bowtie S(B_1, B_2, \dots, B_m)$
 $R.A_i = S.B_j$
 - Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
 - The resulting relation state has one tuple for each combination of tuples— r from R and s from S , but *only if they satisfy the join condition* $r[A_i] = s[B_j]$
 - Hence, if R has n_R tuples, and S has n_S tuples, then the join result will generally have *less than* $n_R * n_S$ tuples.
 - Only related tuples (based on the join condition) will appear in the result



- The general case of JOIN operation is called a Theta-join: $R \bowtie_{\theta} S$

- The join condition is called *theta*
- *Theta* can be any general boolean expression on the attributes of R and S; for example:
 - $R.A_i < S.B_j \text{ AND } (R.A_k = S.B_l \text{ OR } R.A_p < S.B_q)$
- Most join conditions involve one or more equality conditions “AND”ed together; for example:
 - $R.A_i = S.B_j \text{ AND } R.A_k = S.B_l \text{ AND } R.A_p = S.B_q$
- Tuples whose join attributes are NULL or for which the join condition is FALSE do not appear in the result.

BINARY RELATIONAL OPERATIONS: EQUIJOIN

EQUIJOIN Operation

- The most common use of join involves join conditions with *equality comparisons* only
- Such a join, where the only comparison operator used is =, is called an EQUIJOIN.
 - In the result of an EQUIJOIN we always have one or more pairs of attributes (whose names need not be identical) that have identical values in every tuple.
 - The JOIN seen in the previous example was an EQUIJOIN.

Natural join operation

NATURAL JOIN Operation

- Another variation of JOIN called NATURAL JOIN denoted by $*$ was created to get rid of the second (superfluous) attribute in an EQUIJOIN condition.
 - because one of each pair of attributes with identical values is superfluous
- The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, *have the same name* in both relations
- If this is not the case, a renaming operation is applied first.

- **Example:** Combine each PROJECT tuple with the DEPARTMENT tuple that controls the project.

$$\text{PROJ_DEPT} \leftarrow \text{PROJECT} * \rho(\text{Dname}, \text{Dnum}, \text{Mgr_ssn}, \text{Mgr_start_date}) (\text{DEPARTMENT})$$

1 PROJECT

This relation contains project information, for example:

Pname	Pnumber	Dnum
AI	1	5
DBMS	2	4

2 DEPARTMENT

Original DEPARTMENT relation may look like:

Dname	Dnumber	Mgr_ssn	Mgr_start_date
CSE	5	111	2020-01-01
ISE	4	222	2019-06-15

3 ρ (rho) – Rename operator

$\rho(\text{Dname}, \text{Dnum}, \text{Mgr_ssn}, \text{Mgr_start_date}) (\text{DEPARTMENT})$

- ◆ This **renames attributes** of DEPARTMENT
- ◆ Specifically:
 - **Dnumber** → **Dnum**



*

– Natural Join

PROJECT * renamed_DEPARTMENT

- ◆ Natural join automatically matches **common attributes**
- ◆ Here, the common attribute is **Dnum**

5 Result: PROJ_DEPT

The result relation contains **combined information**

Pname	Pnumber	Dnum	Dname	Mgr_ssn	Mgr_start_date
AI	1	5	CSE	111	2020-01-01
DBMS	2	4	ISE	222	2019-06-15

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Location
Headquarters	1	888665555	1981-06-19	Houston
Administration	4	987654321	1995-01-01	Stafford
Research	5	333445555	1988-05-22	Bellaire
Research	5	333445555	1988-05-22	Sugarland
Research	5	333445555	1988-05-22	Houston

DIVISION Operation

- ▶ The Division Operator is used to find tuples in one relation that are related to all tuples in another relation.
- ▶ It's typically used for **"for all"** queries.

Student_Course (Dividend Table):

Student_ID	Course_ID
101	C1
101	C2
102	C1
103	C1
103	C2

Course (Divisor Table):

Course_ID
C1
C2

Example: Query is to find students who are enrolled in all courses listed in the Course table. In this case, students must be enrolled in both C1 and C2.

$$Student_Course_{(Student_ID, Course_ID)} \div Course_{(Course_ID)}$$

Output:

Student_ID
101
103

- **Example:** Retrieve the names of employees who work on all the projects that 'John Smith' works on.

```
JOHN_SSN  $\leftarrow \sigma_{\text{FName}='John' \text{ and } \text{Lname}='Smith'}(\text{EMPLOYEE})$   
JOHN_PNO  $\leftarrow \pi_{\text{Essn}, \text{PNo}}(\text{JOHN\_SSN} \bowtie_{\text{SSN}=\text{ESSN}} \text{WORKS\_ON})$   
SSN_PNUM  $\leftarrow \pi_{\text{Essn}, \text{PNo}}(\text{WORKS\_ON})$   
RES_SSN  $\leftarrow \text{SSN\_PNUM} \div \text{JOHN\_PNO}$   
RESULT  $\leftarrow \pi_{\text{FName}, \text{Lname}}(\text{RES\_SSN} * \text{EMPLOYEE})$ 
```

Step 1

$\text{JOHN_SSN} \leftarrow \sigma_{\text{FName}='John' \text{ and } \text{Lname}='Smith'}(\text{EMPLOYEE})$

Result of step 1

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5

Step 2

$\text{JOHN_PNO} \leftarrow \pi_{\text{Essn}, \text{PNo}} (\text{JOHN_SSN} \bowtie_{\text{SSN}=\text{ESSN}} \text{WORKS_ON})$

JOHN_SSN

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

Result of Step 2

JOHN_PNO

Pno
1
2

Step 3

$SSN_PNUM \leftarrow \pi_{Essn, PNo} (WORKS_ON)$

Result of Step 3

SSN_PNUM

<u>Essn</u>	<u>Pno</u>
123456789	1
123456789	2
666884444	3
453453453	1
453453453	2
333445555	2
333445555	3
333445555	10
333445555	20
999887777	30
999887777	10
987987987	10
987987987	30
987654321	30
987654321	20
888665555	20

Step 4

$\text{RES_SSN} \leftarrow \text{SSN_PNUM} \div \text{JOHN_PNO}$

SSN_PNUM

<u>Essn</u>	<u>Pno</u>
123456789	1
123456789	2
666884444	3
453453453	1
453453453	2
333445555	2
333445555	3
333445555	10
333445555	20
999887777	30
999887777	10
987987987	10
987987987	30
987654321	30
987654321	20
888665555	20

JOHN_PNO

Pno
1
2

Result of Step 4

RES_SSN

Ssn
123456789
453453453

Step 5

RESULT $\leftarrow \pi_{\text{FName, Lname}} (\text{RES_SSN} * \text{EMPLOYEE})$

RES_SSN

Ssn
123456789
453453453

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Final Result

FName	LName
John	Smith
Joyce	English

Aggregate Functions

- ▶ Common functions applied to collections of numeric values include **SUM, AVERAGE, MAXIMUM, and MINIMUM.**
- ▶ The COUNT function is used for counting tuples or values.

- Use of the Aggregate Functional operation $\mathcal{F}$
 - $\mathcal{F}_{\text{MAX Salary}}(\text{EMPLOYEE})$ retrieves the maximum salary value from the EMPLOYEE relation
 - $\mathcal{F}_{\text{MIN Salary}}(\text{EMPLOYEE})$ retrieves the minimum Salary value from the EMPLOYEE relation
 - $\mathcal{F}_{\text{SUM Salary}}(\text{EMPLOYEE})$ retrieves the sum of the Salary from the EMPLOYEE relation
 - $\mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}}(\text{EMPLOYEE})$ computes the count (number) of employees and their average salary
 - Note: count just counts the number of rows, without removing duplicates

USING GROUPING WITH AGGREGATION

- Grouping can be combined with Aggregate Functions
- Example: For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY
- A variation of aggregate operation $\mathcal{F}$ allows this:
 - Grouping attribute placed to left of symbol
 - Aggregate functions to right of symbol
 - $\text{DNO } \mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}} (\text{EMPLOYEE})$
- Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department

EMPLOYEE

SSN	Name	Salary	DNO
1	Asha	30000	10
2	Ravi	40000	10
3	Neha	50000	20
4	Arun	60000	20
5	Kiran	55000	20

Department 10

- Employees: Asha, Ravi
- $\text{COUNT}(\text{SSN}) = 2$
- $\text{AVG}(\text{Salary}) = (30000 + 40000) / 2 = 35000$

Department 20

- Employees: Neha, Arun, Kiran
- $\text{COUNT}(\text{SSN}) = 3$
- $\text{AVG}(\text{Salary}) = (50000 + 60000 + 55000) / 3 = 55000$

Final Result

DNO	COUNT(SSN)	AVG(Salary)
10	2	35000
20	3	55000

■ **Recursive Closure Operations**

- Another type of operation that, in general, cannot be specified in the basic original relational algebra is **recursive closure**.
 - This operation is applied to a **recursive relationship**.
- An example of a recursive operation is to retrieve all SUPERVISEES of an EMPLOYEE e at all levels — that is, all EMPLOYEE e' directly supervised by e ; all employees e'' directly supervised by each employee e' ; all employees e''' directly supervised by each employee e'' ; and so on.

- **Example:** Specify the SSN of all the employees directly supervised at level one by 'James Borg'.

$$\text{BORG_SSN} \leftarrow \pi_{\text{Ssn}} (\sigma_{\text{Fname}='James' \text{ AND } \text{Lname}='Borg'} (\text{EMPLOYEE}))$$
$$\text{SUPERVISION} (\text{SSN1}, \text{SSN2}) \leftarrow \pi_{\text{Ssn}, \text{Super_ssn}} (\text{EMPLOYEE})$$
$$\text{RESULT} \leftarrow \pi_{\text{SSN1}} (\text{SUPERVISION} \bowtie \text{BORG_SSN})$$
$$\text{SSN2}=\text{Ssn}$$

(Borg's SSN is 888665555)

(SSN)

(SUPERSSN)

SUPERVISION	SSN1	SSN2
	123456789	333445555
	333445555	888665555
	999887777	987654321
	987654321	888665555
	666884444	333445555
	453453453	333445555
	987987987	987654321

RESULT 1	SSN
	333445555
	987654321

(Supervised by Borg)

RESULT 2	SSN
	123456789
	999887777
	666884444
	453453453
	987987987

(Supervised by Borg's subordinates)

RESULT	SSN
	123456789
	999887777
	666884444
	453453453
	987987987
	333445555
	987654321

(RESULT1 $\cup$ RESULT2)

OUTER JOIN

Outer Join is based on both matched and unmatched data. Outer Joins subdivide further into,

1. Left Outer Join
2. Right Outer Join
3. Full Outer Join

Left Outer Join ($A \bowtie B$)

Left Outer Join returns all the rows from the table on the left even if no matching rows have been found in the table on the right. When no matching record is found in the table on the right, NULL is returned.



Consider the following 2 Tables

A	
Num	Square
2	4
3	9
4	16
B	
Num	Cube
2	8
3	18
5	75

A ⋈ B			
A ⋈ B			
Num	Square	Cube	
2	4	8	
3	9	18	
4	16	-	

Right Outer Join ($A \bowtie B$)

Right Outer Join returns all the columns from the table on the right even if no matching rows have been found in the table on the left. Where no matches have been found in the table on the left, NULL is returned. RIGHT outer JOIN is the opposite of LEFT JOIN

In our example, let's assume that you need to get the names of members and movies rented by them. Now we have a new member who has not rented any movie yet.



$A \bowtie B$

A ⋈ B		
Num	Cube	Square
2	8	4
3	18	9
5	75	-

Full Outer Join ($A \bowtie B$)

In a **Full Outer Join** , all tuples from both relations are included in the result, irrespective of the matching condition.

Example:

$A \bowtie B$

$A \bowtie B$

Num	Square	Cube
2	4	8
3	9	18
4	16	–
5	–	75